

卷积码

peterrolls@qq.com

2026 年 5 月 30 日

1 定义

卷积码是一种纠错编码技术，属于信道编码的一种。

与分组码不同，卷积码的编码过程具有记忆性：每个时刻的输出不仅取决于当前的输入，还与之前若干时刻的输入相关。这种特性通过位移寄存器实现。

卷积码是线性码。

卷积码通过以下参数描述：

输入长度	输出长度	码率	约束长度
k	n	$R = k/n$	$K = v + 1$

表 1: 卷积码参数

其中 v 是寄存器级数，卷积码编码时如有多组寄存器， v 指其中最长的那一组的级数。**约束长度**定义为每个输入比特影响输出的最大时间跨度。

2 编码

2.1 多项式表示

设有 k 路序列输入： $u_0(D), u_1(D), \dots, u_{k-1}(D)$ ；以及 n 路序列输出： $c_0(D), c_1(D), \dots, c_{n-1}(D)$ 。“输入——输出”关系通过一个生成矩阵 $G(D)$ 来关联：

$$G(D) = (g_{i,j}(D)) = \begin{bmatrix} g_{0,0}(D) & g_{0,1}(D) & \cdots & g_{0,n-1}(D) \\ g_{1,0}(D) & g_{1,1}(D) & \cdots & g_{1,n-1}(D) \\ \vdots & \vdots & \ddots & \vdots \\ g_{k-1,0}(D) & g_{k-1,1}(D) & \cdots & g_{k-1,n-1}(D) \end{bmatrix}$$

其中 $g_{i,j}(D)$ 是有理函数形式，即两个多项式相除的形式。

传统卷积码的 n 和 k 都比较小（个位数），记：

$$C(D) = [c_0(D), c_1(D), \dots, c_{n-1}(D)]$$

$$U(D) = [u_0(D), u_1(D), \dots, u_{k-1}(D)]$$

则卷积码编码可表示为： $C(D) = U(D)G(D)$ 。对 $G(D)$ 经初等变化进行标准化可得到系统码的生成矩阵。

2.2 矩阵表示

生成矩阵也可以表示成普通线性码的矩阵形式。

将输入矢量 U 使用线性无关的基元素构建，然后依次位移输入到生成矩阵 $G(D)$ 中，直至达到码长长度，其输出按行拼接就是生成矩阵。

以例子说明。卷积码：

$$G(D) = \begin{bmatrix} 1+D & D & 1+D \\ D & 1 & 1 \end{bmatrix}$$

输入 $u_0(t)$ 、 $u_1(t)$ 构造为：

$$u_0 u_1 : 10$$

$$u_0 u_1 : 01$$

首先将 $u_0 u_1 = (10)$ 输入，即：

$$\begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} 1+D & D & 1+D \\ D & 1 & 1 \end{bmatrix} = \begin{bmatrix} 1+D & D & 1+D \end{bmatrix}$$

得到：

$$c_0(t) = 110 \quad 000 \quad 000 \quad \dots$$

$$c_1(t) = 010 \quad 000 \quad 000 \quad \dots$$

$$c_2(t) = 110 \quad 000 \quad 000 \quad \dots$$

$$c = 101 \quad 111 \quad 000 \quad \dots$$

然后将 $u_0 u_1 = (01)$ 输入，即：

$$\begin{bmatrix} 0 & 1 \end{bmatrix} \begin{bmatrix} 1+D & D & 1+D \\ D & 1 & 1 \end{bmatrix} = \begin{bmatrix} D & 1 & 1 \end{bmatrix}$$

得到：

$$c_0(t) = 010 \quad 000 \quad 000 \quad \dots$$

$$c_1(t) = 100 \quad 000 \quad 000 \quad \dots$$

$$c_2(t) = 100 \quad 000 \quad 000 \quad \dots$$

$$c = 011 \quad 100 \quad 000 \quad \dots$$

将上面得到的两个输入的输出写到一起：

$$\begin{bmatrix} 1 & 0 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 & 0 & 0 \end{bmatrix}$$

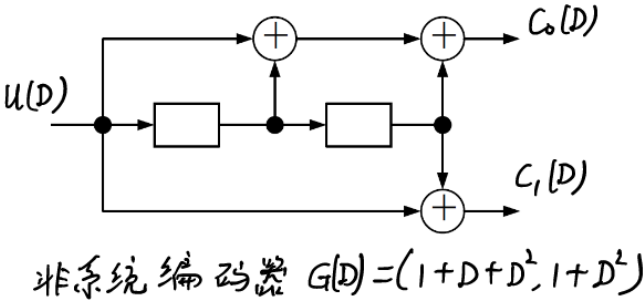
进一步得到生成矩阵 G ：

$$G = \begin{bmatrix} 1 & 0 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 & 0 & 0 \\ & & 1 & 0 & 1 & 1 & 1 & 1 \\ & & 0 & 1 & 1 & 1 & 0 & 0 \\ & & & & 1 & 0 & 1 & 1 & 1 & 1 \\ & & & & 0 & 1 & 1 & 1 & 0 & 0 \\ & & & & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \end{bmatrix}$$

由生成矩阵 G 也可以产生校验矩阵 H 。

2.3 网格图表示

网格图表示类似状态机，表示编码器中寄存器值的“当前状态”、“输入”、“输出”和“下一状态”。使用例子说明。对于如下图所示的(2,1,2)，生成多项式矩阵 $G(D) = [1 + D + D^2, 1 + D^2]$ 的卷积码：

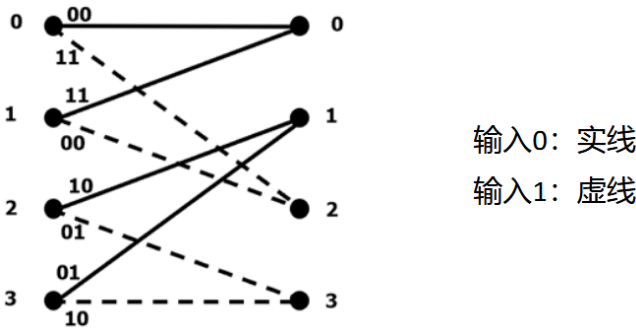


在不同输入，不同状态的情况下计算输出，得到以下的状态转移关系表：

当前状态	输入	输出	下一状态
$s_1 s_0(t)$	u	$c_0 c_1$	$s_1 s_0(t + 1)$
00	0	00	00
00	1	11	10
01	0	11	00
01	1	00	10
10	0	10	01
10	1	01	11
11	0	01	01
11	1	10	11

表 2: 状态转移关系表

可以用网格图来表示上述表格：



对于归零卷积码，从图的左上角00开始编码。

3 维特比译码

3.1 最小自由距离

假设卷积码的码字 $\vec{v}$ 和 $\vec{v}'$ 具有有限长度，起始且终止于全零状态 s_0 ，其信息序列 $\vec{u}$ 和 $\vec{u}'$ 分别对应码字 $\vec{v}$ 和 $\vec{v}'$ 。在信息序列上分别附加 m 个收尾分组。

卷积码的最小自由距离定义为：

$$d_{\text{free}} \triangleq \min_{\vec{u}', \vec{u}''} \{d(\vec{u}', \vec{u}'') : \vec{u}' \neq \vec{u}''\}$$

最小自由距离是卷积码中所有可能的码字序列之间最小的汉明距离。这个距离越大，卷积码的纠错能力就越强，因为它可以区分更多的错误模式，从而在接收端更准确地恢复原始数据。

同时， d_{free} 也是由任意有限长度的非零信息序列产生的码字的最小重量。在归零卷积码中，它也是网格图中从零状态分叉再合并于零状态的所有有限长度路径的最小汉明重量。

3.2 分支度量

衡量接收序列片段与候选分支输出的差异。

3.3 路径度量

从初始状态到当前状态的某条路径的**累计差异值**。等于路径上所有分支度量的累加。

维特比译码通过保留每个状态的**最小路径度量**来降低复杂度。

常见的度量方法有汉明距离、欧氏距离和对数似然函数。

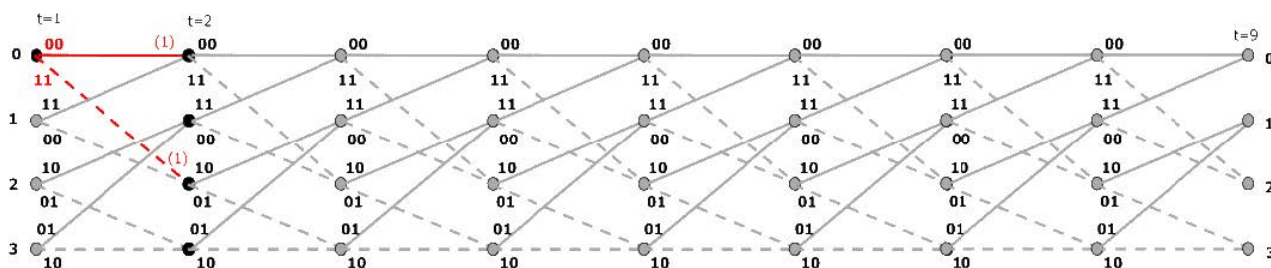
3.4 译码步骤

1. 初始化：假设编码器从全0状态开始，初始路径度量为0，时间步 $t = 0$ ；
2. 逐时刻处理($t = 1, 2, \dots, T$):
 - (a) 计算分支度量：对当前时刻的每个状态，计算从上一时刻可能转移来的所有分支的度量；
 - (b) 更新路径度量：遍历所有可能的上一时刻状态，选择最小的候选路径度量作为新路径度量，并记录对应的幸存路径；
 - (c) 裁剪非最优路径：相同终点的状态只保留一条最优路径；
3. 回溯：以编码器在全零状态结束，从最终时刻的最小度量状态反向追踪到初始状态，得到解码序列。

以一个归零卷积码硬判决Viterbi译码为例：

编码器输出：11 10 00 01 01 00 10 11

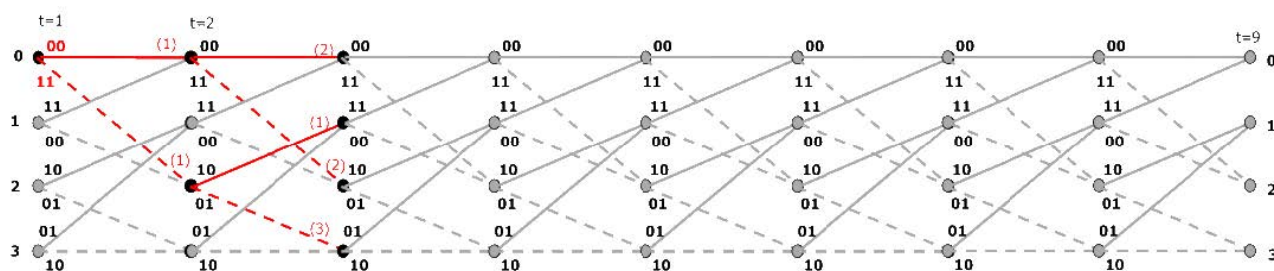
信道输出：10 10 00 01 11 00 11 11



时刻1(10)：从状态0出发，输入“0”和“1”，转移到下一状态（时刻2(10)），计算支路度量和路径度量：

路径0-0的度量：1

路径0-2的度量：1



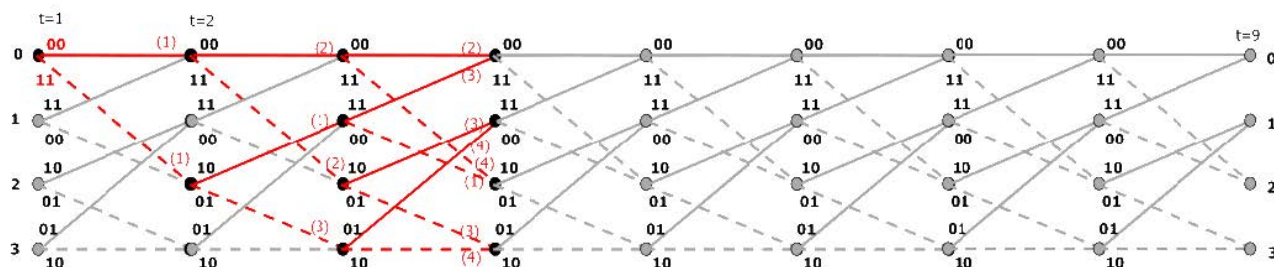
时刻2(10): 从状态0出发, 输入“0”和“1”, 转移到下一状态(时刻3(00)), 计算之路度量 and 路径度量:

路径0-0-0的度量: 2

路径0-2-1的度量: 1

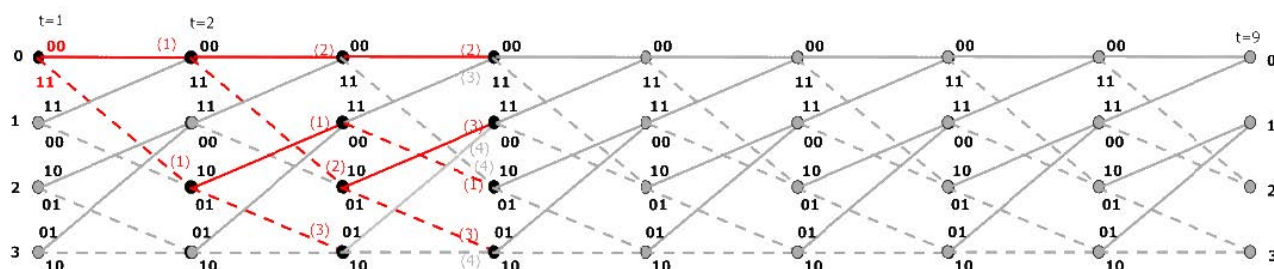
路径0-0-2的度量: 2

路径0-2-3的度量: 3



时刻3(00): 从状态0出发, 输入“0”和“1”, 转移到下一状态(时刻4(01)), 计算之路度量 and 路径度量。

第3时刻共有4个状态, 每一条状态如图示有两条输入路径。对于终止于同一状态的多条路径, 只保留其中路径度量较小的一条路径(路径度量相同时, 任选其中一条保留)。裁剪后, 幸存的路径如下:



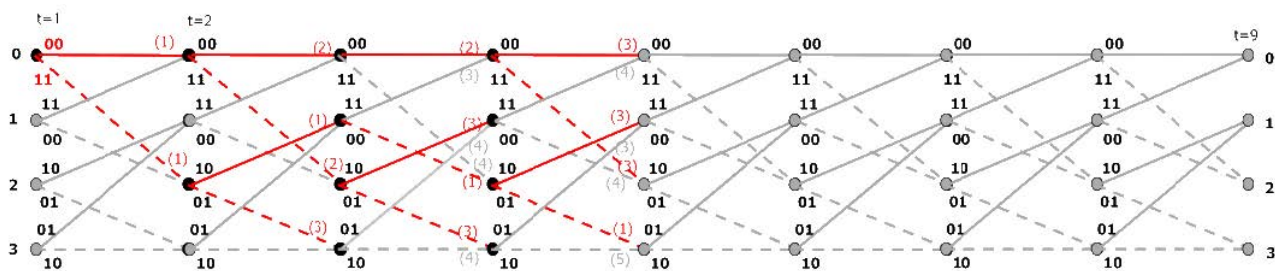
路径0-0-0-0的度量: 2

路径0-0-2-1的度量: 3

路径0-2-1-3的度量: 1

路径0-0-2-3的度量: 3

时刻4(01): 从状态0出发, 输入“0”和“1”, 转移到下一状态(时刻5(11)), 计算之路度量 and 路径



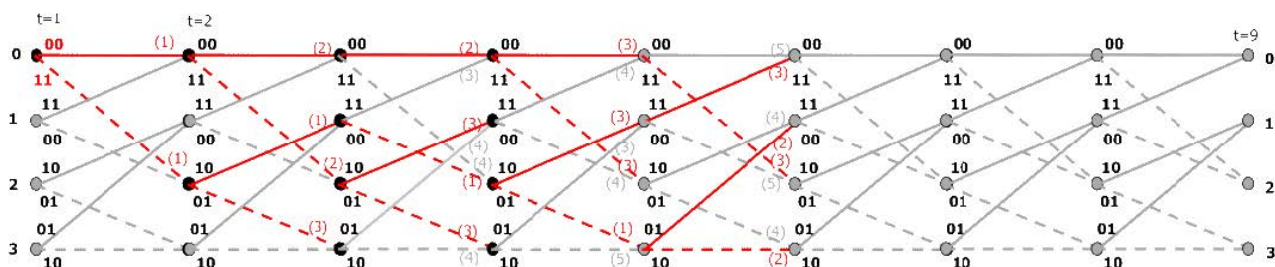
度量。第4时刻共有4个状态，按上一步进行裁剪，幸存路径：

路径0-0-0-0-0的度量：3

路径0-2-1-2-1的度量：3

路径0-0-0-0-2的度量：3

路径0-2-1-2-3的度量：1



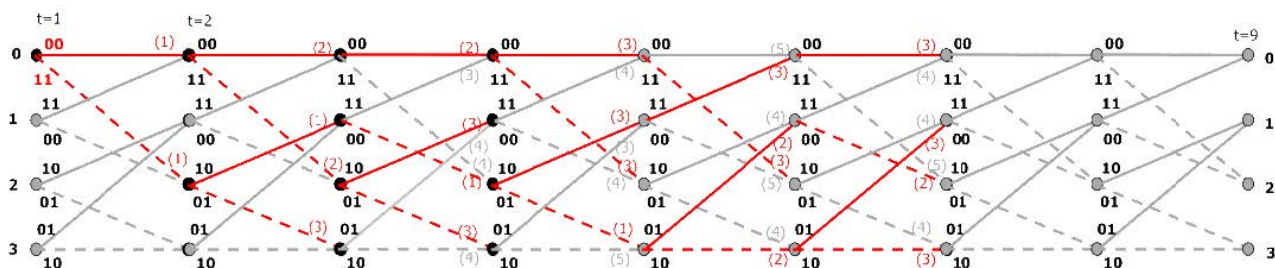
时刻5(11)：从状态0出发，输入“0”和“1”，转移到下一状态（时刻6(00)），计算之路度量 and 路径度量。幸存路径：

路径0-2-1-2-1-0的度量：3

路径0-2-1-2-3-1的度量：2

路径0-0-0-0-0-3的度量：3

路径0-2-1-2-3-3的度量：2



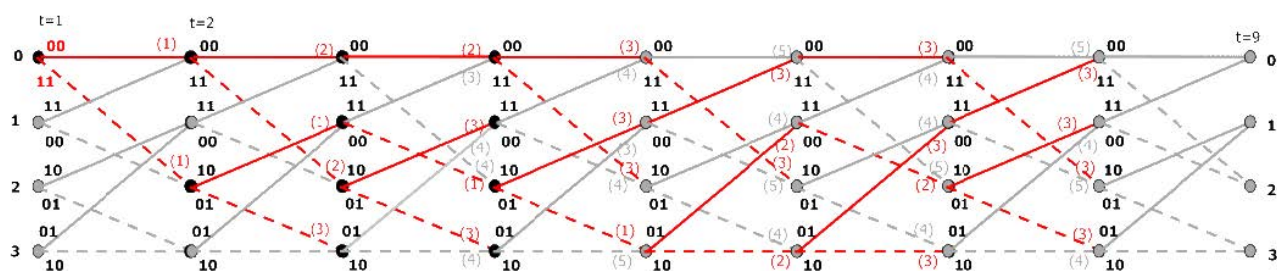
时刻6(00)：从状态0出发，输入“0”和“1”，转移到下一状态（时刻7(11)），计算之路度量 and 路径度量。幸存路径：

路径0-2-1-2-1-0-0的度量：3

路径0-2-1-2-3-3-1的度量：3

路径0-2-1-2-3-1-2的度量：2

路径0-2-1-2-3-3-3的度量：3



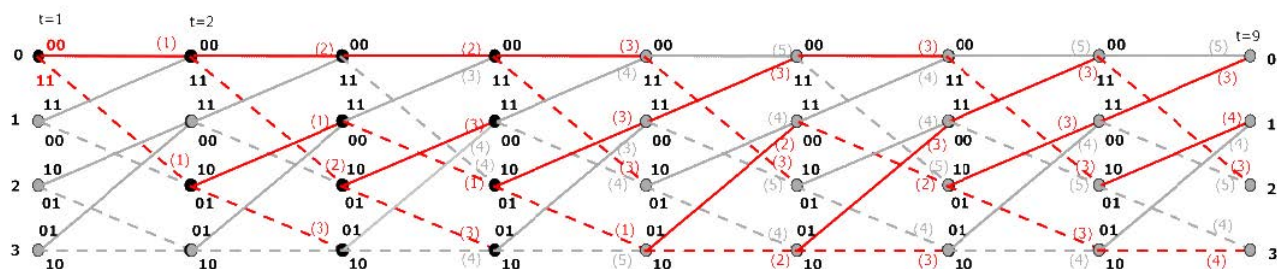
时刻7(11): 从状态0出发, 输入“0”和“1”, 转移到下一状态(时刻8(11)), 计算之路度量和路径度量。幸存路径:

路径0-2-1-2-3-3-1-0的度量: 3

路径0-2-1-2-3-1-2-1的度量: 3

路径0-2-1-2-1-0-0-2的度量: 3

路径0-2-1-2-3-1-2-3的度量: 3



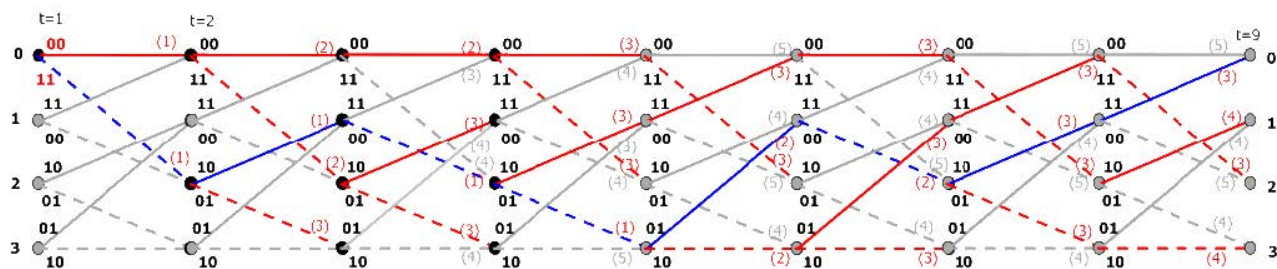
时刻8(11): 从状态0出发, 输入“0”和“1”, 转移到下一状态(时刻9(00)), 计算之路度量和路径度量。幸存路径:

路径0-2-1-2-3-1-2-1-0的度量: 3

路径0-2-1-2-1-0-0-2-1的度量: 4

路径0-2-1-2-3-3-1-0-2的度量: 3

路径0-2-1-2-3-1-2-3-3的度量: 4



回溯到最佳路径: 0 - 2 - 1 - 2 - 3 - 1 - 2 - 1 - 0

译码输出: 1110000101001011

发送端编码输出: 1110000101001011

与发送端编码输出一致, 译码完成。

例子中给定的是硬判决译码（即计算汉明距离）。改变度量计算方法可得到软判决译码——如欧氏距离。对于非归零卷积码，可以采用3到5倍约束长度的延迟，进行边输入边输出的译码。

维特比译码小结：

1. 所有编码路径均起始于0状态；
2. 起始于0状态，在中间时刻 t 终止于同一状态的多条路径，保留其中度量最小的一条路径；
3. 如果卷积码编码状态归零，在终点时刻仅存在一条幸存路径。从“0”状态回溯得到最终译码输出路径、最大似然码序列和信息序列；
4. 译码器主要操作：加法、比较、选择、回溯；
5. 幸存路径数与状态数相同，译码复杂度与状态数呈正比，与码长呈线性关系。

3.5 渐近编码增益

起始于网格图上某个 t 时刻的0状态，终止于大于 t 时刻的0状态的所有编码路径对应的码序列，重量为 d_{free} （最小自由距离）的码序列数。记为 $A_{d_{\text{free}}}$ 。

上述例子中， $A_{d_{\text{free}}} = 1$ 。

4 BCJR译码

BCJR译码方法也是Turbo码译码的核心算法。

已知卷积码的输入输出及状态转移关系，收到 $[y_1, y_2, \dots, y_\tau]$ 。设编码器状态数为 s_T ，译码步骤如下：

1. 计算分支度量 $r_t(m', m)$ $t = 1, 2, \dots, \tau$ ；
2. 设置： $\alpha_0(m)$ ，当 $m = 0$ 时， $\alpha_0(m) = 1$ ，当 $m \neq 0$ 时， $\alpha_0(m) = 0$ 。迭代计算 $\alpha_t(m)$ ， $t = 1, 2, \dots, \tau$ ， $m = 0, 1, \dots, s_T - 1$ ， $\alpha_t(m) = \sum_{m'} \alpha_{t-1}(m') r_t(m', m)$ ；
3. 设置： $\beta_0(m)$ ，当 $m = 0$ 时， $\beta_0(m) = 1$ ，当 $m \neq 0$ 时， $\beta_0(m) = 0$ 。迭代计算 $\beta_t(m)$ ， $t = \tau - 1, \tau - 2, \dots, 1$ ， $m = 0, 1, \dots, s_T - 1$ ， $\beta_t(m) = \sum_z \beta_{t+1}(z) r_{t+1}(m, z)$ ；
4. 计算 $\lambda_t(m)$ 及 $\sigma_t(m', m)$ ：
 - (a) $\lambda_t(m) = \alpha_t(m) \beta_t(m)$ ；
 - (b) $\sigma_t(m', m) = \alpha_{t-1}(m') r_t(m', m) \beta_t(m)$ ， $t = 1, 2, \dots, \tau$ ；
5. 计算 $P\{u_t = a; y_1^\tau\} = \sum_{m', m} P(u_t = a | s_{t-1} = m'; s_t = m) \sigma_t(m', m)$ ；
6. 归一化得到 $P\{y_1^\tau\}$ 从而有 $P\{u_t = a | y_1^\tau\} = P\{u_t = a; y_1^\tau\} / P\{y_1^\tau\}$ ，并判决输出 $\hat{u}_t$ 。也可以直接观察 t 时刻使 $P\{u_t = a; y_1^\tau\}$ 最大的 a 。

优化方法Log-MAP算法，将上述的 α_t 改为 $\log \alpha_t$ 、 β_t 改为 $\log \beta_t$ 、 $r_t(m', m)$ 改为 $\log r_t(m', m)$ 。此处不列举。